Computer and Network Security (2026-2027)

Part 3: Cryptographic Hash Functions

Jeroen Famaey
jeroen.famaey@uantwerpen.be

University of Antwerp
Faculty of Science

Part 3 overview

1. Hash Function Basics
2. Secure Hash Function (SHA)
   - SHA-1 and SHA-2
   - Length Extension Attack
   - SHA-3
3. Message Authentication
   - Message Authentication Basics
   - Message Authentication Codes
   - Authenticated Encryption
4. Secure Password Storage

Hash Function Basics

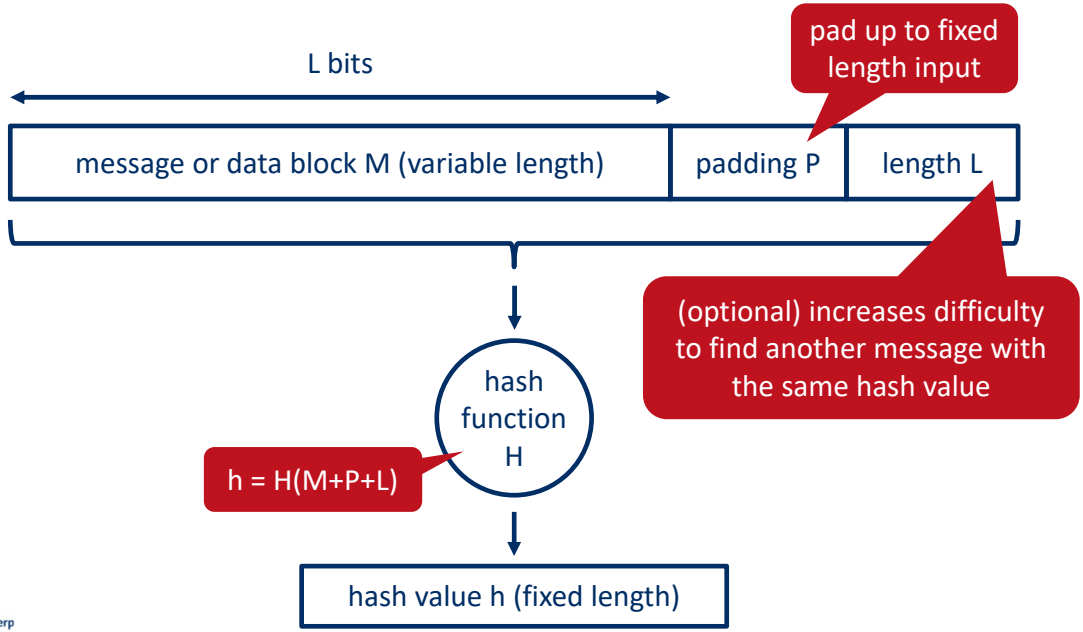
117 million LinkedIn password hashes leaked in 2016

- The passwords were hashed using unsalted SHA-1
- It took hackers a mere 2 hours on a computing cluster to hack 65%

Frequency	Hash	Plaintext
1135936	7c4a8d09ca3762af61e59520943dc26494f8941b	123456
207488	7728240c80b6bfd450849405e8500d6d207783b6	linkedin
188380	5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8	password
149916	f7c3bc1d808e04732adf679965ccc34ca7ae3441	123456789
95854	7c222fb2927d828af22f592134e8932480637c0d	12345678

Source: *https://blog.korelogic.com/blog/2016/05/19/linkedin_passwords_2016*

What is a hash function?



Applications of cryptographic hash functions

Message authentication	Assures that the data received was not altered or replayed
Digital signatures	Assures both the authenticity of the message and identity of the sender
One-way password file	Stores a hash value of the password rather than the password itself
Intrusion/virus detection	Store H(F) for each file to determine if files have been modified
Pseudorandom function	Use the hash function to generate pseudorandom private keys

Security requirements

variable input size	Input data can be of any size	basic
Fixed output size	Output is of fixed length	
Efficiency	H(x) is easy to compute	
Preimage resistant	Given h, it is hard to find y: H(y) = h	advanced
Second preimage resistant	Given x, it is hard to find y: $y \neq x$ & H(y) = H(x)	
Collision resistant	It is hard to find (x, y): H(x) = H(y)	

The first three properties are required for any practical hash function
The last three extend security and can exist together or independently

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

Which applications of cryptographic hash functions require second preimage resistance?

0

- Message authentication
- Digital signatures
- Intrusion and virus detection
- One-way password file

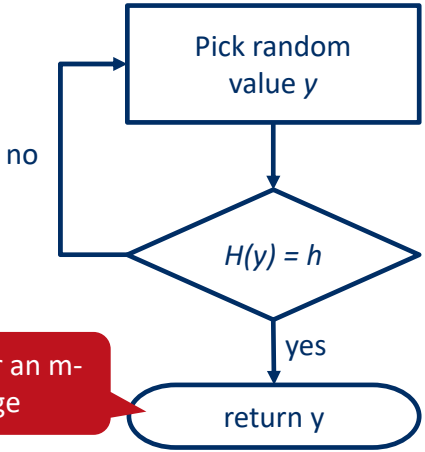
Security requirements of applications

	Preimage resistant	Second pre-image resistant	collision resistant
message authentication	Yes	Yes	Yes*
digital signatures	Yes	Yes	Yes*
intrusion and virus detection		Yes	
one-way password file	Yes		

*Only required against a chosen plaintext message attack

Brute force preimage attacks

- Circumvent the (second) preimage resistance properties
- **Goal:** For a given h, find y such that H(y) = h (or for a given x, find y such that H(y) = H(x))

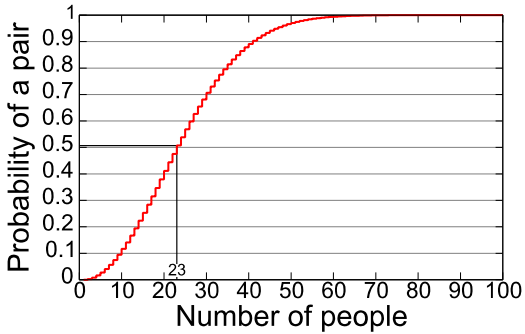


2^{m-1} attempts are needed for an m-bit hash value on average

Brute force collision resistance attacks

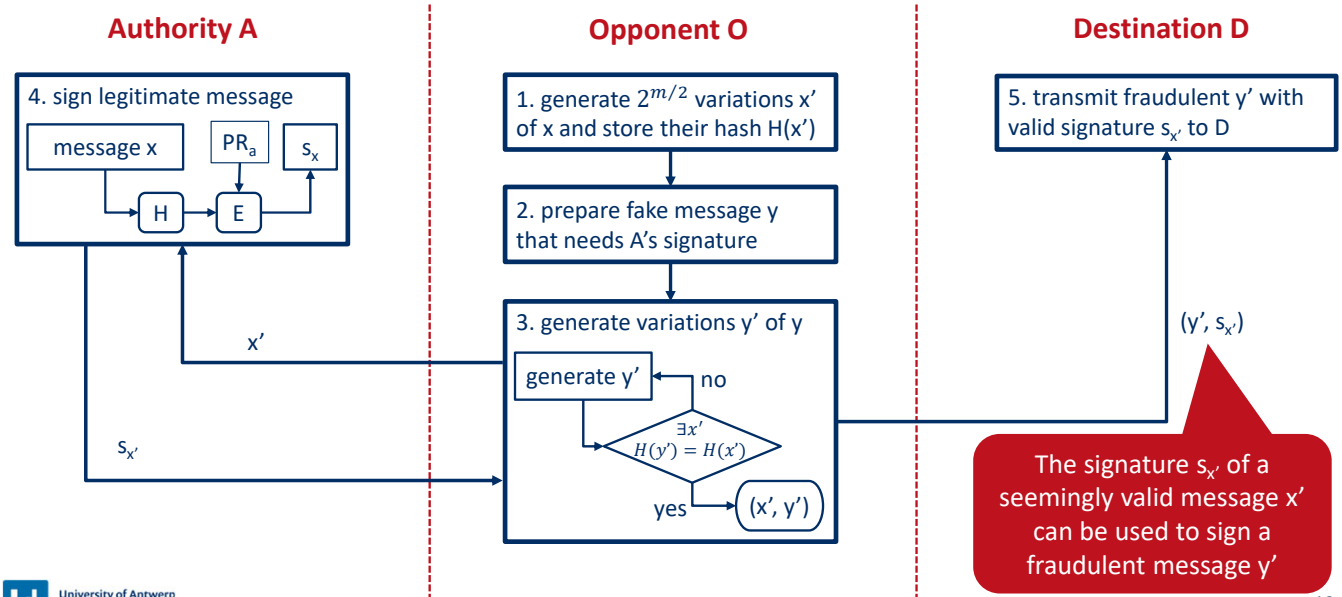
- Circumvent the collision resistance properties
- **Goal:** Find x and y such that H(x) = H(y)
- Using the birthday paradox, it can be shown that on average only $\sqrt{2^m} = 2^{m/2}$ attempts are needed (for an m-bit hash value)

- **Birthday paradox**
  - Among 23 people, there is a >50% chance that two share a birthday
  - **Generalization:** Choosing random variables from a uniform distribution in the range of 0 to N – 1, the probability of a repeated element exceeds 0.5 after ~ $\sqrt{N}$ choices have been made



Source: *Wikipedia, © Rajkiran g, CC BY-SA 3.0*

Collision resistance attack using birthday paradox



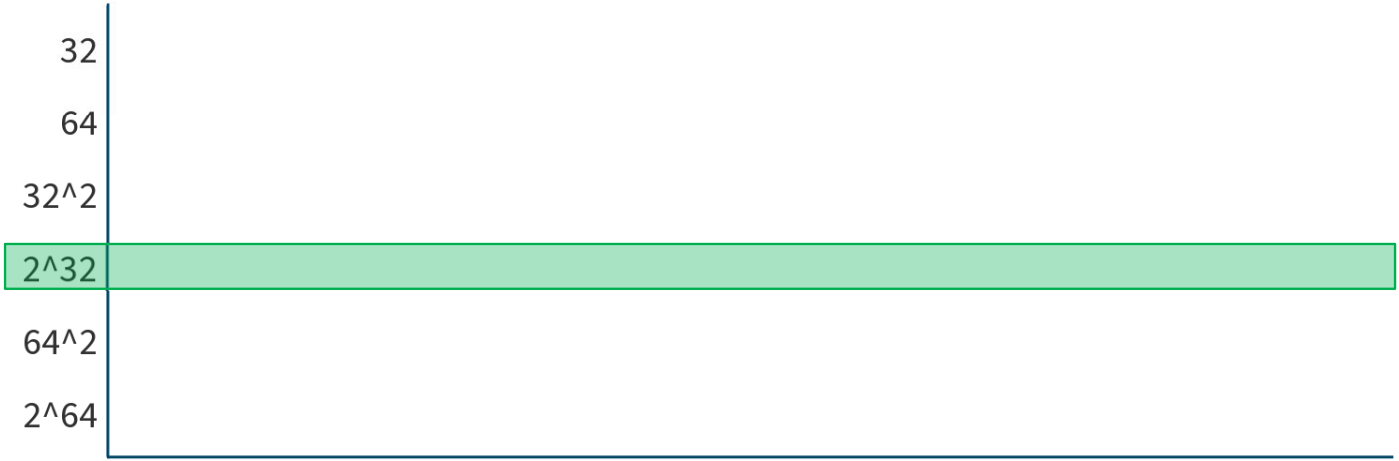
When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

Imagine a 33 word text, where each word separator can be 1 or 2 spaces long, how many semantically equivalent variations of the text can be created?

0

- 32
- 64
- 32^2
- 2^32
- 64^2
- 2^64

Imagine a 33 word text, where each word separator can be 1 or 2 spaces long, how many semantically equivalent variations of the text can be created?



For a 64-bit hash only 2^{32} variations are needed

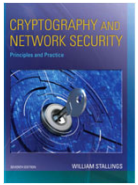
As { the / — } Dean of Blakewell College, I have { had the pleasure of knowing / known } Cherise Rosetti for the { last / past } four years. She { has been / was } { a tremendous / an outstanding } { asset to / role model in } { our / the } school. I { would like to take this opportunity to / wholeheartedly } recommend Cherise for your { school's / — } graduate program. I { am / feel } { confident / certain } { that / — } { she / Cherise } will { continue to / — } succeed in her studies. { She / Cherise } is a dedicated student and { thus far her grades / her grades thus far } { have been / are } { exemplary / excellent }. In class, { she / Cherise } { has proven to be / has been } a take-charge { person / individual } { who is / — } able to successfully develop plans and implement them.

{ She / Cherise } has also assisted { us / — } in our admissions office. { She / Cherise } has { successfully / — } demonstrated leadership ability by counseling new and prospective students. { Her / Cherise's } advice has been { a great / of considerable } help to these students, many of whom have { taken time to share / shared } their comments with me regarding her pleasant and { encouraging / reassuring } attitude. { For these reasons / It is for these reasons that } I { highly recommend / offer high recommendations for } Cherise { without reservation / unreservedly }. Her { ambition / drive } and { abilities / potential } will { truly / surely } be an { asset to / plus for } your { establishment / school }.

These are 2^{38} variations of the same message already

Summary: Hash Function Basics

- **Cryptographic hash functions** transform variable length input into fixed length hash code
- Many **applications** in computer security
  - Message authentication
  - Digital signatures
  - One-way password files
  - Intrusion & virus detection
  - Pseudorandom private key generation
- Efficient chosen plaintext attacks using **birthday paradox** are possible



Link with the book

- Chapter 11 (Sections 11.1, 11.3)

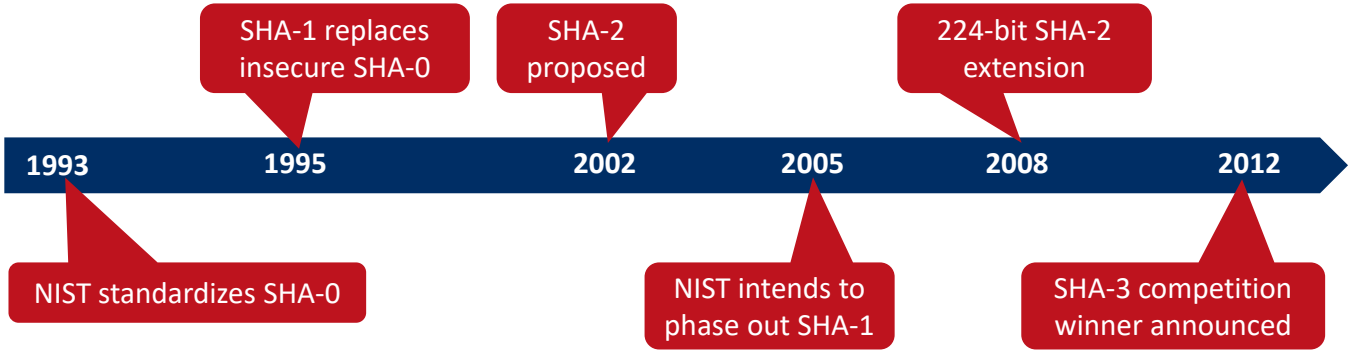


Link with the videos

- Week 3 (Collision Resistance 1)

Secure Hash Function (SHA)

Secure Hash Algorithm (SHA) history



1. SHA-1 and SHA-2
2. Length Extension Attack
3. SHA-3

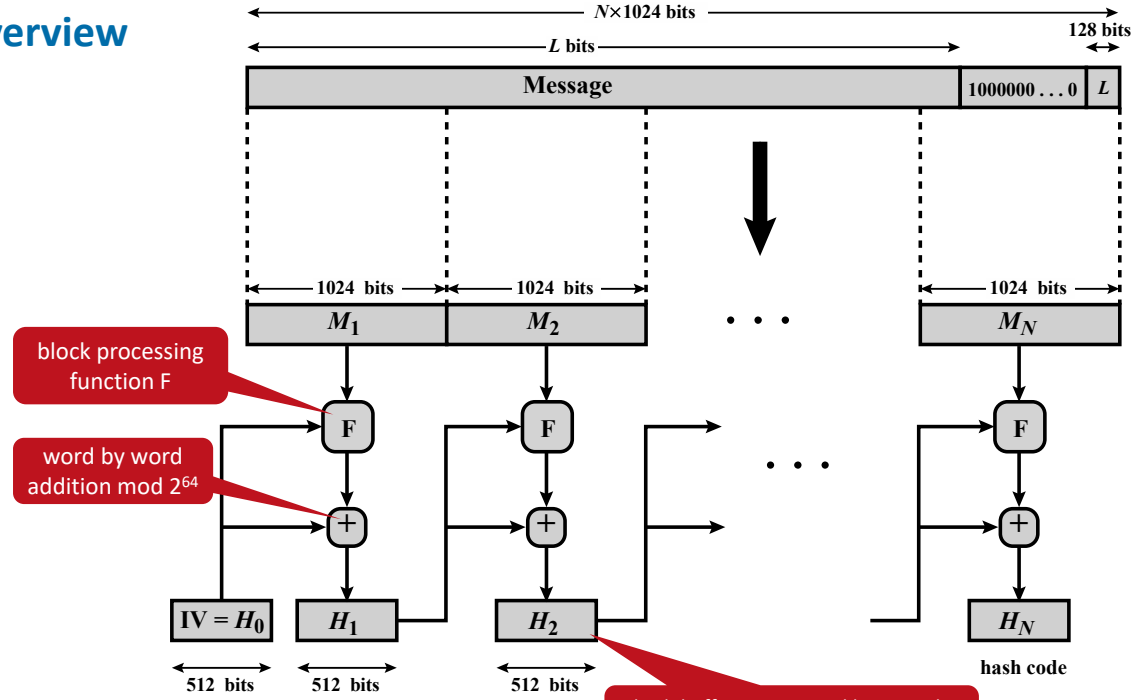
The original SHA hash function family (sizes in bits)

- SHA-1 and SHA-2 use the same structure as SHA-0 and MD4
- SHA-224 is generated using SHA-256, truncated to the 224 leftmost bits
- SHA-384, SHA-512/224 and SHA-512/256 use t leftmost bits of SHA-512
- SHA-1 and SHA-2 are **susceptible to length extension attacks**

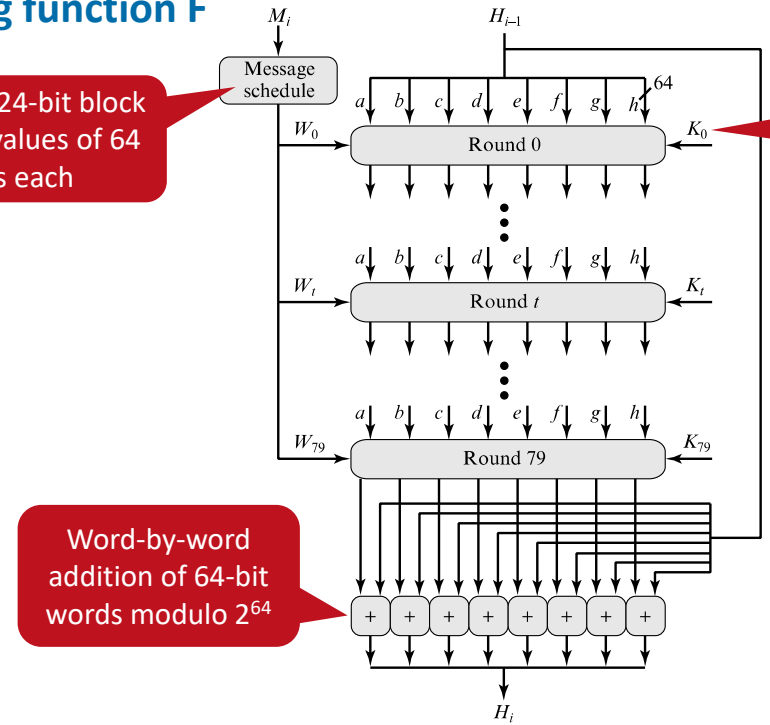
	SHA-1	SHA-2					
		SHA-224	SHA-256	SHA-384	SHA-512	SHA-512/224	SHA-512/256
Hash size*	160	224	256	384	512	224	256
Message size*	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$
Block size*	512	512	512	1024	1024	1024	1024
Word size*	32	32	32	64	64	64	64
Rounds	80	64	64	80	80	80	80

* in bits

SHA-512 overview



SHA-512 block processing function F

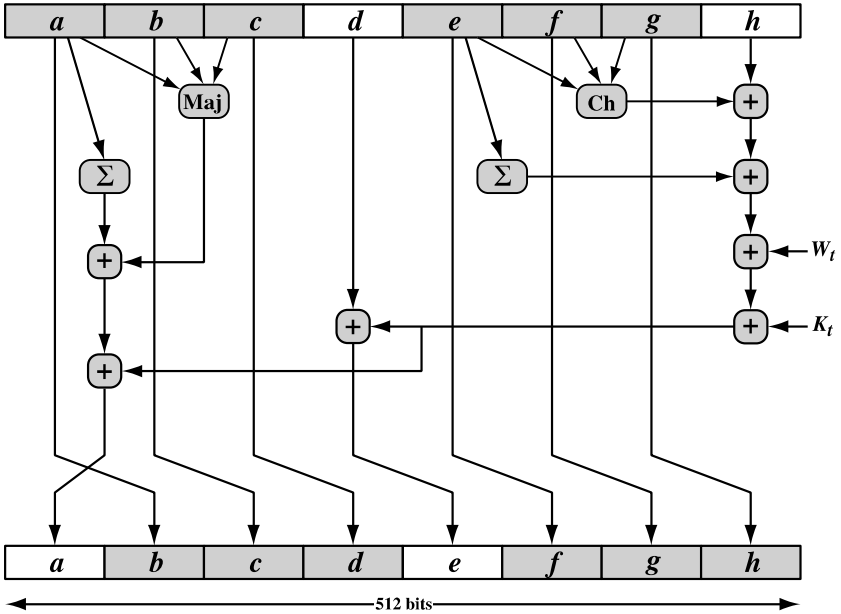


Turns 1024-bit block into 80 values of 64 bits each

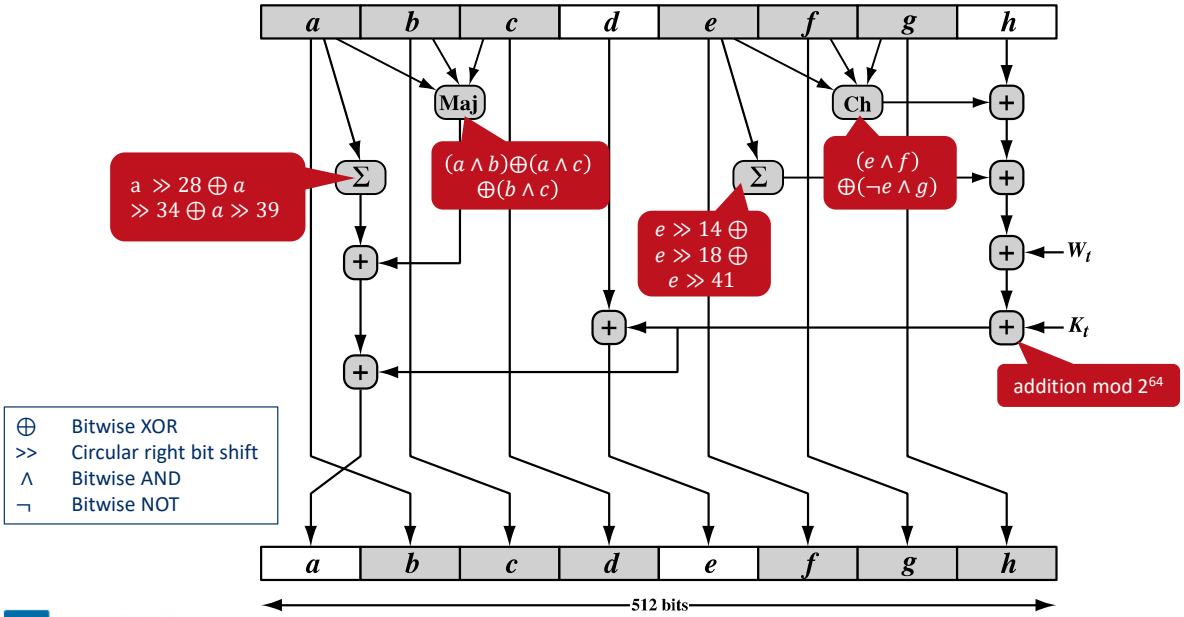
64-bit constant

Word-by-word addition of 64-bit words modulo 2^{64}

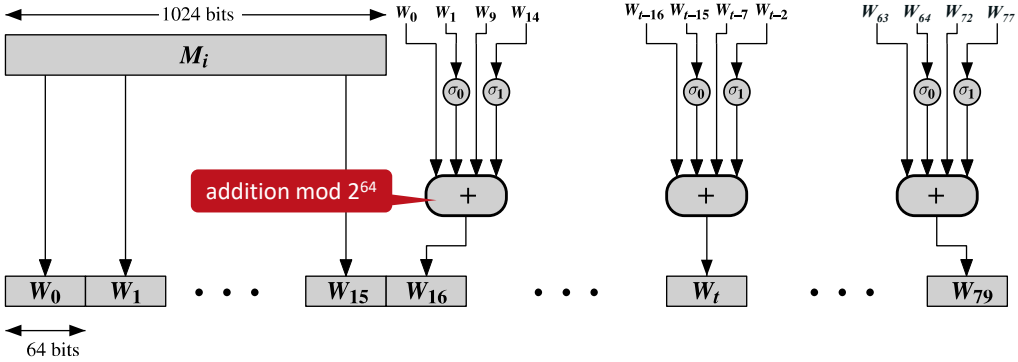
Round function: substitution and permutation



Round function: substitution and permutation



Transforming a block M into 64-bit words W_0-W_{79}



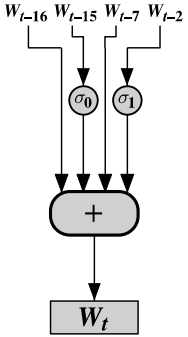
$$\sigma_0(x) = \mathrm{ROTR}^1(x) \oplus \mathrm{ROTR}^8(x) \oplus \mathrm{SHR}^7(x)$$
$$\sigma_1(x) = \mathrm{ROTR}^{19}(x) \oplus \mathrm{ROTR}^{61}(x) \oplus \mathrm{SHR}^6(x)$$

$\mathrm{ROTR}^n(x)$ = circular right bit-shift of the 64-bit argument x by n bits
$\mathrm{SHR}^n(x)$ = right bit-shift of the 64-bit argument x by n bits with padding by n zeros on the left
$\oplus$ = bitwise XOR

Exercise: Calculating SHA round words

Exercise: Given a simplified version of SHA with a 128-bit block size and 8-bit word size, calculate round word W_{16}.

M = {5C 84 C2 03 06 9A AF A7 5A CA 8D 4B C9 2A 06 30}



$$\sigma_0(x) = \mathrm{ROTR}^1(x) \oplus \mathrm{ROTR}^2(x) \oplus \mathrm{SHR}^7(x)$$
$$\sigma_1(x) = \mathrm{ROTR}^3(x) \oplus \mathrm{ROTR}^5(x) \oplus \mathrm{SHR}^6(x)$$

$\mathrm{ROTR}^n(x)$ = circular right bit-shift of the 64-bit argument x by n bits
$\mathrm{SHR}^n(x)$ = right bit-shift of the 64-bit argument x by n bits with padding by n zeros on the left

Solution: Calculating SHA round words (1/2)

$$W_{16} = W_0 + \sigma_0(W_1) + W_9 + \sigma_1(W_{14}) \; mod \; 2^8$$
$$W_{16} = 5C + \left(\text{ROTR}^1(84) \oplus \text{ROTR}^2(84) \oplus \text{SHR}^7(84)\right) + CA$$
$$+ \left(\text{ROTR}^3(06) \oplus \text{ROTR}^5(06) \oplus \text{SHR}^6(06)\right) \; mod \; 2^8$$

84	= 1000 0100	06	= 0000 0110
$\text{ROTR}^1(84)$	= 0100 0010	$\text{ROTR}^3(06)$	= 1100 0000
$\text{ROTR}^2(84)$	= 0010 0001	$\text{ROTR}^5(06)$	= 0011 0000
$\text{SHR}^7(84)$	= 0000 0001	$\text{SHR}^6(06)$	= 0000 0000
---	---	---	---
$\oplus$	= 0110 0010	$\oplus$	= 1111 0000
	= 62		= F0

Solution: Calculating SHA round words (2/2)

$W_{16} = W_0 + \sigma_0(W_1) + W_9 + \sigma_1(W_{14}) \; mod \; 2^8$

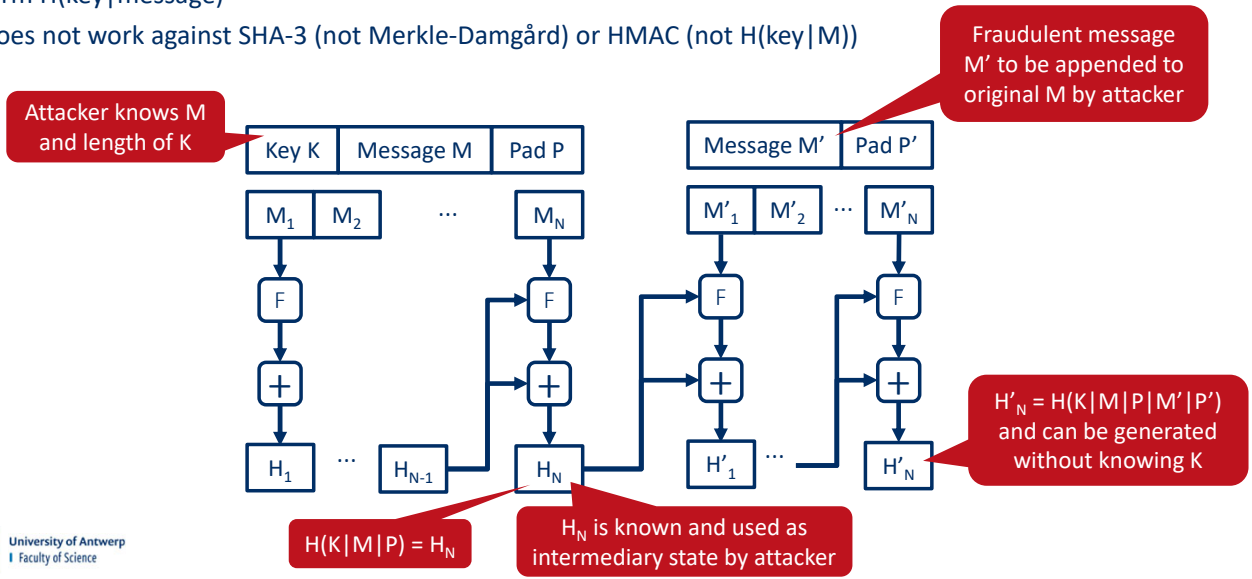
W_0	0101 1100	=	{5C}	=	92
$\sigma_0(W_1)$	0110 0010	=	{62}	=	98
W_9	1100 1010	=	{CA}	=	202
$\sigma_1(W_{14})$	1111 0000	=	{F0}	=	240
+				=	632
$mod \; 2^8$					120

$W_{16} = 120 = \{78\}$

1. SHA-1 and SHA-2
2. Length Extension Attack
3. SHA-3

Length extension attack

- Works against any Merkle-Damgård hash (e.g., SHA-1, SHA-2), used as a message authentication code (MAC) of the form H(key|message)
- Does not work against SHA-3 (not Merkle-Damgård) or HMAC (not H(key|M))



Length extension attack: HTTP example

- Consider a website for ordering waffles online
- The user orders a waffle of type eggo online using the following request URL:
  - Original Data: count=10&lat=37.351&user_id=1&long=-119.827&waffle=eggo
  - Original Signature: 6d5f807e23db210bc254a28be2d6759a0f5f5d99

- An attacker can overwrite choice of waffle by overwriting the URL parameter:
  - Desired New Data: count=10&lat=37.351&user_id=1&long=-119.827&waffle=eggo**&waffle=liege**

- If the attacker knows the length of key K, and the algorithm used, they can generate the **correct padding** from the original request and append **new data**:
  - New Data: count=10&lat=37.351&user_id=1&long=-119.827&waffle=eggo
    \x80\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
    \x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
    \x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
    \x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x02\x**&waffle=liege**
  - New Signature: 0e41270260895979317fff3898ab85668953aaa2

Source: *https://en.wikipedia.org/wiki/Length_extension_attack*

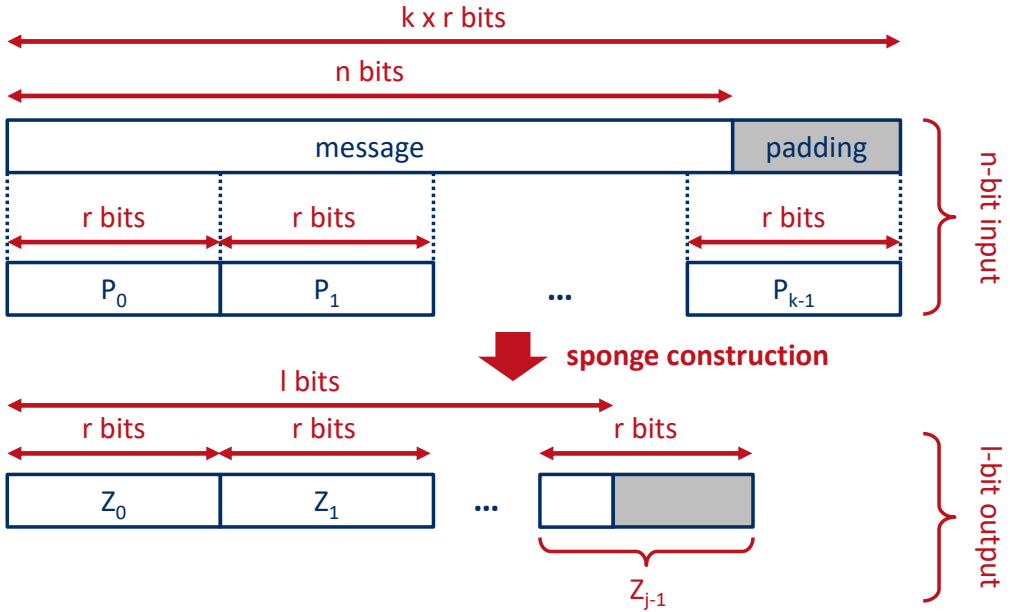
1. SHA-1 and SHA-2
2. Length Extension Attack
3. SHA-3

SHA-3 parameters

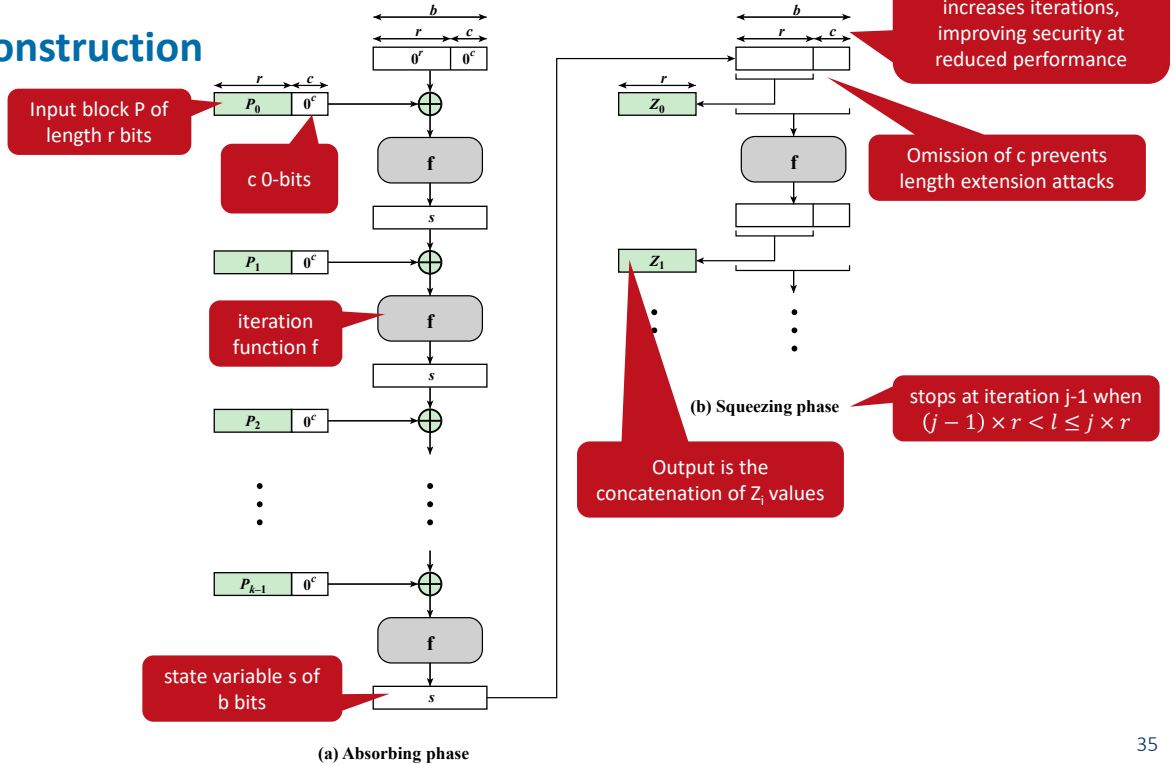
	SHA-3			
Hash code size (bits)	224	256	384	512
Message size	--	--	--	--
Block size r (bits)	1152	1088	832	576
Word size (bits)	64	64	64	64
Number of rounds	24	24	24	24
Capacity c	448	512	768	1024
Collision resistance	2^{112}	2^{128}	2^{192}	2^{256}
Second preimage resistance	2^{224}	2^{256}	2^{384}	2^{512}

$r + c = 1600$ bits

SHA-3 uses variable input/output sponges

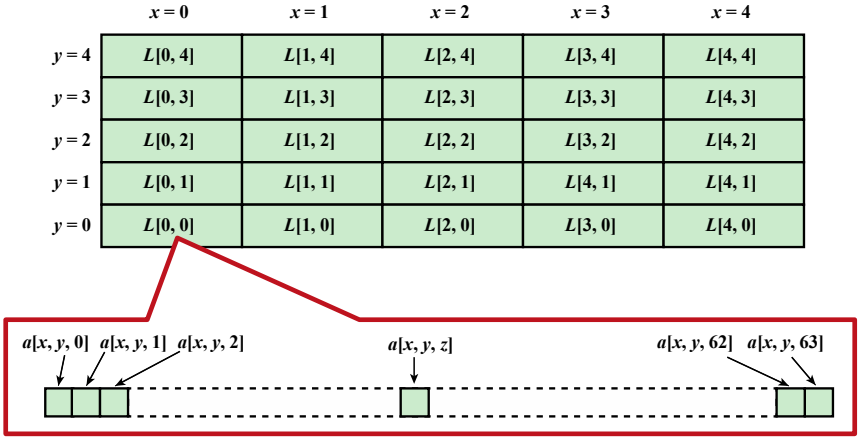


Sponge construction

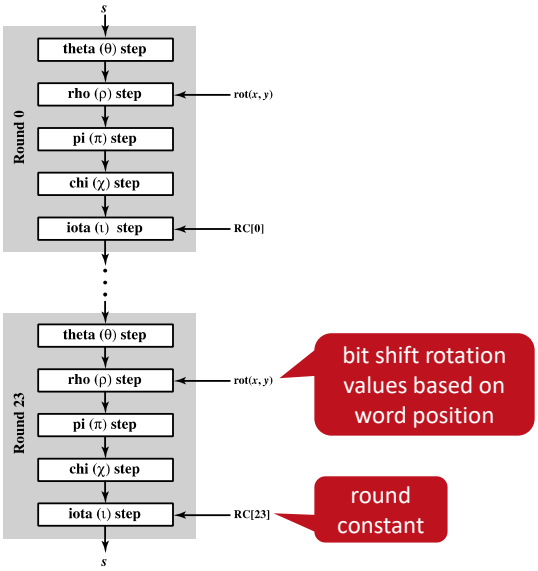


SHA-3 uses Keccak-f as iteration function f

- By default, SHA-3 uses variables r and c, with $r + c = 1600$ bits
- Internal state is represented by 5x5 matrix of 64-bit “lanes” or “words”
  - a[x, y, z] refers to bit z of the lane at column x and row y
  - L[x, y] refers to the lane at column x and row y



Iteration function f has 24 rounds



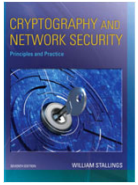
Function	Type	Description
θ	Substitution	New value of each bit in each word depends on its current value and on one bit in each word of preceding column and one bit of each word in succeeding column.
ρ	Permutation	The bits of each word are permuted using a circular bit shift. $W[0, 0]$ is not affected.
π	Permutation	Words are permuted in the 5×5 matrix. $W[0, 0]$ is not affected.
χ	Substitution	New value of each bit in each word depends on its current value and on one bit in next word in the same row and one bit in the second next word in the same row.
ι	Substitution	$W[0, 0]$ is updated by XOR with a round constant.

rot(x, y):

	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$
$y = 4$	18	2	61	56	14
$y = 3$	41	45	15	21	8
$y = 2$	3	10	43	25	39
$y = 1$	36	44	6	55	20
$y = 0$	0	1	62	28	27

Summary: SHA

- **SHA-1 and SHA-2** are based on Merkle-Damgård principle, where each output block is used as the initial state for the next input block
- Merkle-Damgård hash functions are vulnerable to **length extension attacks** if used as a message authentication code of the form H(Key|Message)
- **SHA-3** is a more secure successor to SHA-2
  - It uses a capacity parameter to only reuse part of each output block as input for the next
  - The capacity reduces vulnerability to the length extension attack



Link with the book

- Chapter 11 (Sections 11.5, 11.6)



Link with the videos

- Week 3 (Collision Resistance 2)

Message Authentication

1. Message Authentication Basics
2. Message Authentication Codes
3. Authenticated Encryption

Types of attacks against transmitted messages

1. Disclosure of message content
2. Traffic analysis

confidentiality

3. Masquerade
4. Content modification
5. Sequence modification
6. Timing modification

message authentication

7. Source repudiation

digital signatures

8. Destination repudiation

Definitions

Message authentication

Verify that received messages have not been altered (i.e., no modification, insertion, deletion, or replay).

Digital signatures

An authentication technique that also includes measures to counter repudiation by the source.

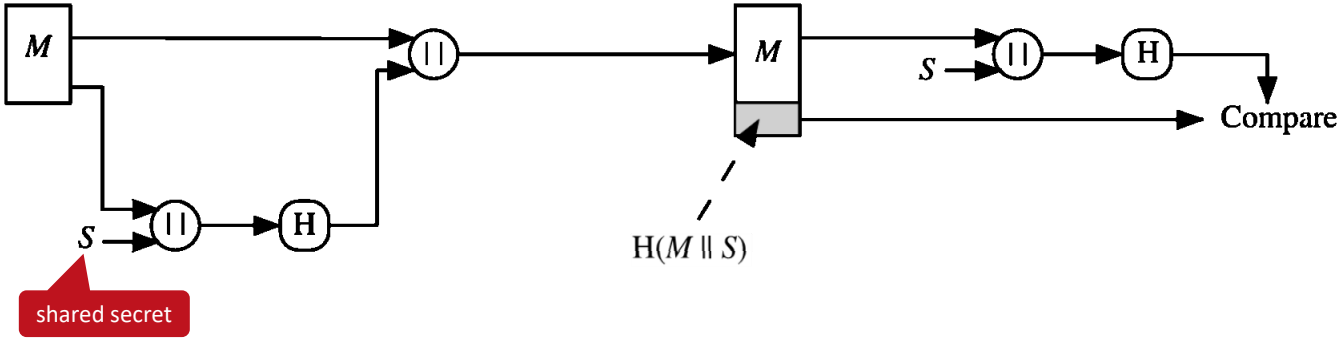
Message authentication functions

= function that produces an authenticator: a value to be used to authenticate a message.

3 classes

- **Hash function:** Fixed length hash value serves as authenticator
- **Message encryption:** Ciphertext serves as authenticator
- **Message Authentication Code (MAC):** Fixed-length value resulting from message and secret key serves as authenticator

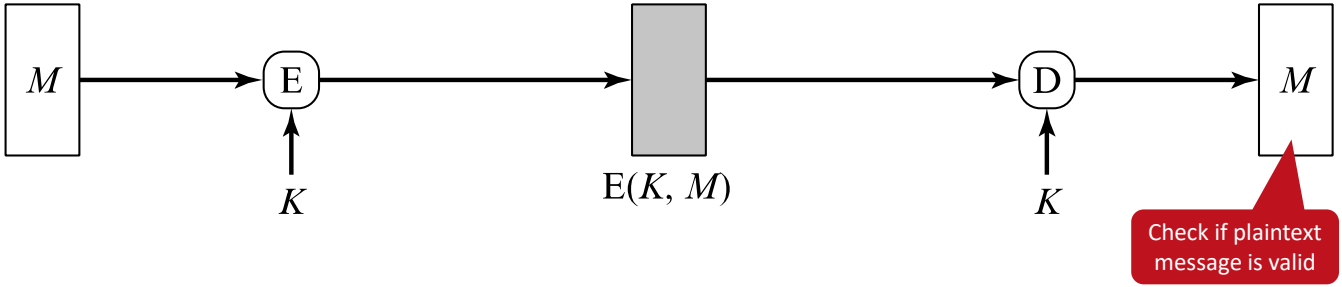
Using hash functions for message authentication



Vulnerable to Length Extension Attack if used in combination with SHA-1 or SHA-2 based hashing algorithm!

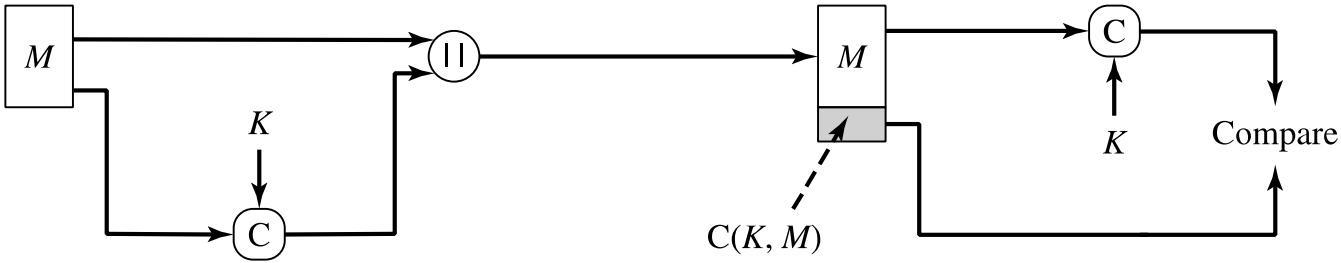
Using symmetric encryption for authentication

- Provides both confidentiality and authentication if K is secret
- If the decrypted message M is not meaningful (e.g., random bits) then the message can be considered altered
- Some automated means to determine legitimate plaintext may be needed
- Cannot be used if encryption algorithm is **malleable** (e.g., stream ciphers, or block operation modes such as CBC (partially) and CTR)



Message authentication code (MAC)

- Alternative method to provide message authentication
- Generates a fixed-size **cryptographic checksum** based on the message and a secret key
- Does **not** need to be **reversible**, in contrast to encryption



When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

Can a message authentication code (MAC) be used as a digital signature?

0

- Yes
- No
- It depends

Security requirements for MAC functions

Given a MAC function, and an opponent who knows the MAC, but does not know the secret key K, the following requirements should be satisfied:

1. If an opponent observes M and $MAC(K, M)$ it should be infeasible to construct M', such that $MAC(K, M') = MAC(K, M)$

2. Given two randomly chosen messages M and M', the probability that $MAC(K, M) = MAC(K, M')$ should be 2^{-n} (for an n-bit code)

3. If M' is a known transformation of M (i.e., $M' = f(M)$), then $Pr[MAC(K, M) = MAC(K, M')] = 2^{-n}$

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

Given a MAC function with key size k bits and code size n (< k) bits, what is the computational effort required to brute force the key given a single message-code pair?

0

- 2^k
- 2^n
- The key cannot be unambiguously determined

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

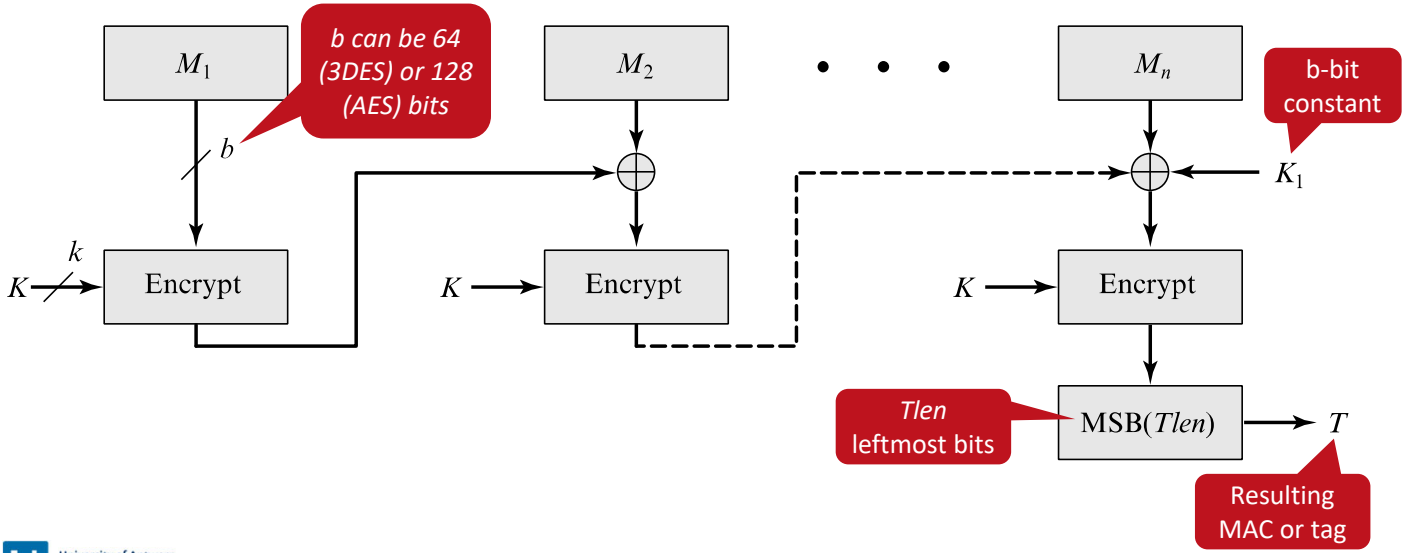
Given a key size k of 24 bits and code size n of 16 bits, how many text-code pairs are needed on average to determine the key?

0

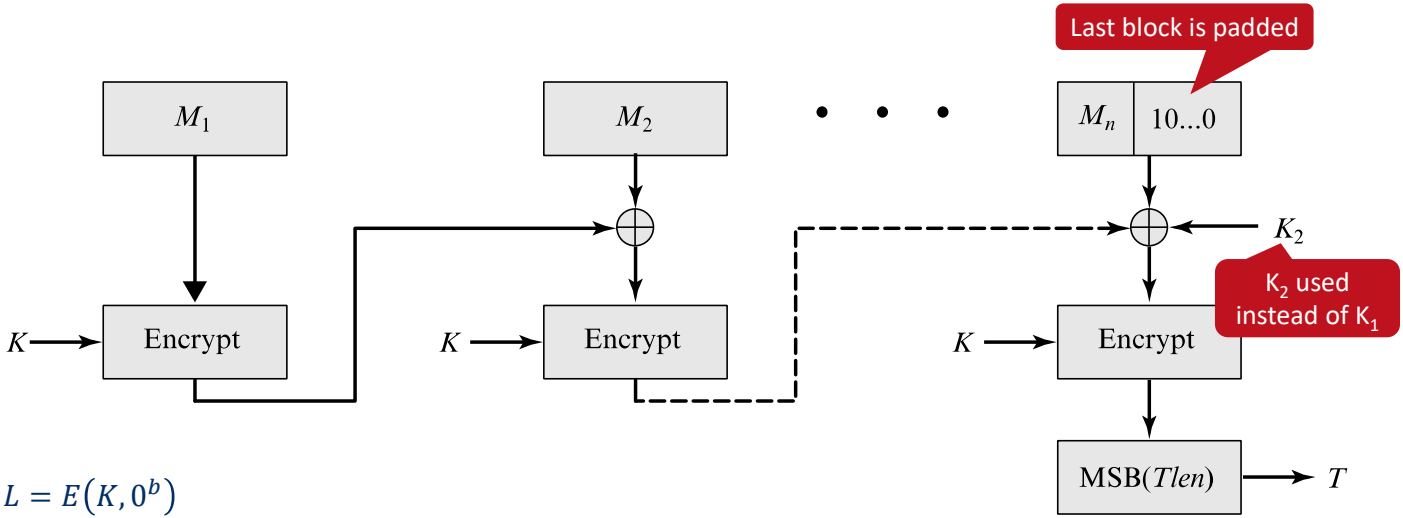
- 1
- 2
- 3
- 4
- 5

1. Message Authentication Basics
2. Message Authentication Codes
3. Authenticated Encryption

Cipher-Based Message Authentication Code (CMAC) if len(M_n) = b



CMAC if message length is not an integer multiple of b



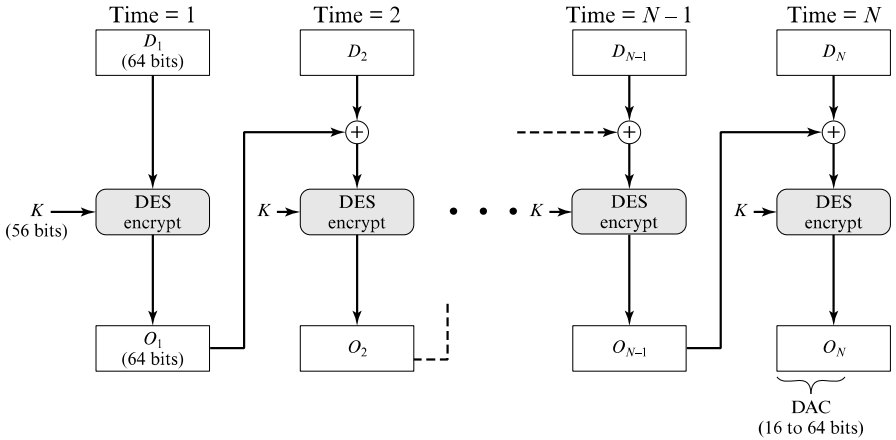
$L = E(K, 0^b)$

$K_1 = L \cdot x$ (Multiplication in GF(2^b))

$K_2 = L \cdot x^2$

Why the need for two different constants K_1 and K_2?

- The Data Authentication Algorithm (DAA), CMAC's predecessor, does not use these constants, which makes it insecure unless only messages of one fixed integer-multiple of the block size are processed
- As a simple example, for a single block message X, the following equation holds when using DAA: $MAC(\mathrm{K}, X) = T = MAC(\mathrm{K}, X||X \oplus T)$



Hash-function based MAC: HMAC

- Several **advantages** compared to traditional MACs, based on symmetric block ciphers:
  1. Cryptographic hash functions, e.g., SHA, generally execute faster
  2. Library code for hash functions is widely available

- However, there are some issues to overcome as hash functions are not designed to **rely on a secret key**

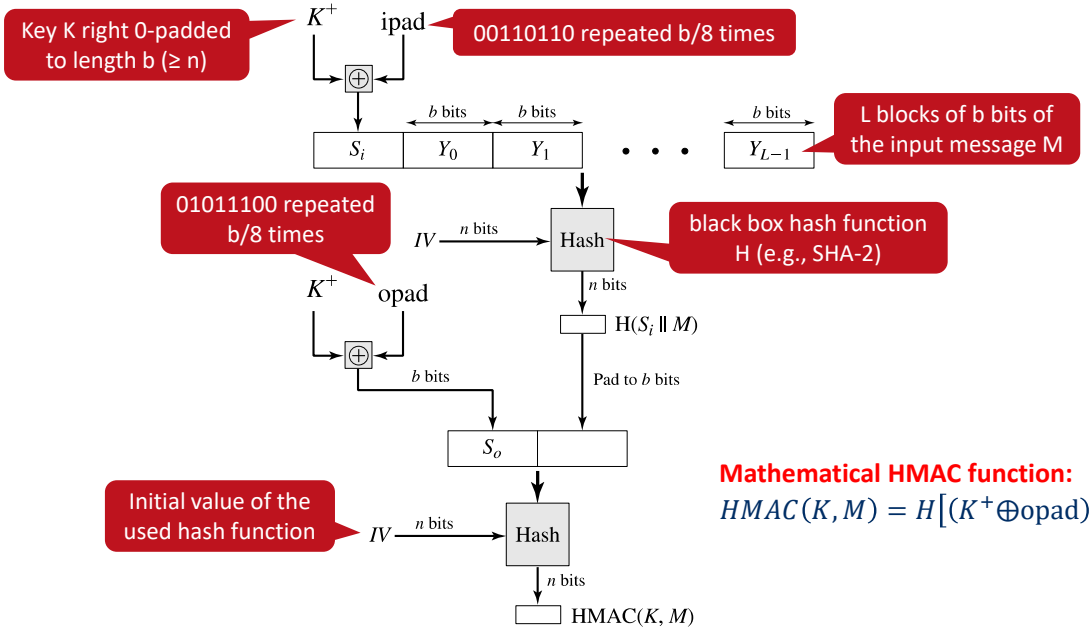
- HMAC was first published in 1996 and is used for IP security (IPSec), TLS and SSL authentication

HMAC design objectives

RFC 2104 lists 5 objectives for HMAC:

1. To use, without modification, available hash functions
2. To allow for easy replaceability of the hash function
3. To preserve the hash function's original performance
4. To use and handle keys in a simple way
5. To have a well understood cryptographic analysis

HMAC can be used with any hash function



Mathematical HMAC function:

$$HMAC(K, M) = H\left[(K^+ \oplus \text{opad}) || H[(K^+ \oplus \text{ipad}) || M]\right]$$

Cryptographic strength of HMAC

- HMAC can be proven secure provided that the embedded hash function has some reasonable cryptographic strengths

- A successful attack on HMAC is equivalent to one of the following attacks on the hash function:
  - The attacker is able to compute an output of the compression function (with secret and unknown IV)
  - The attacker finds collisions in the hash function

- The second attack requires $2^{n/2}$ attempts (i.e., birthday attack)
  - Not trivial as message-code pairs cannot be generated offline (as attacker does not know K)
  - On a 1 Gbps link, using MD5 (128-bit hash code, 512-bit block size), this would take ±300.000 years without a change in key

Exercise: How long does it take to break HMAC with SHA-256



Exercise: Calculate how many years it would take to find a collision in HMAC making use of SHA-256 (256-bit hash code, 512-bit block size) with data transmitted over a 1 Gbps link.

1 Message Authentication Basics

2 Message Authentication Codes

3 Authenticated Encryption

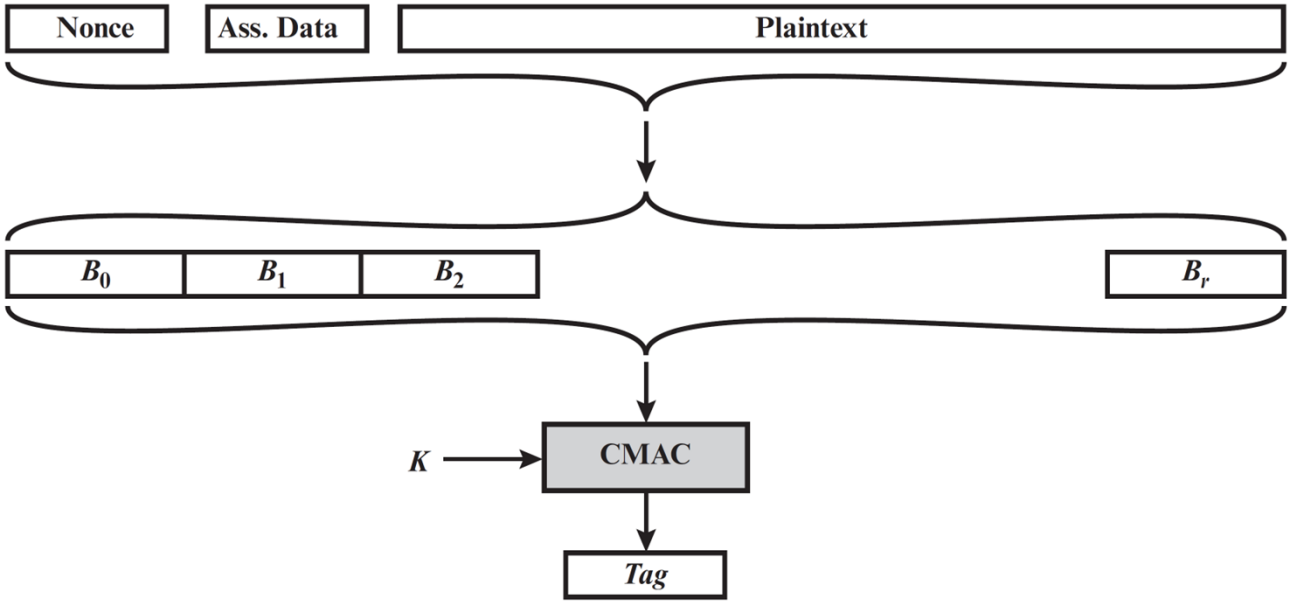
Authenticated Encryption (AE)

- **Definition:** Encryption systems that simultaneously protect confidentiality and authenticity (integrity)
- Four main methods
  - Hashing followed by encryption (H → E): $C = E(K, M||H(M))$
    - Used by Wi-Fi WEP → **Was shown to have fundamental flaws!**
  - Authentication followed by encryption (A → E): $C = E\big(K_2, M||MAC(K_1, M)\big)$
    - Used by SSL/TLS
  - Encryption followed by authentication (E → A): $C = E(K_2, M), T = MAC(K_1, C)$
    - Used by IPSec
  - Independently encrypt and authenticate (E + A) : $C = E(K_2, M), T = MAC(K_1, M)$
    - Used by SSH
- Two approaches for AE have been standardized by NIST
  - Counter with Cipher Block Chaining-Message Authentication Code (CCM)
  - Galois/Counter Mode (GCM)

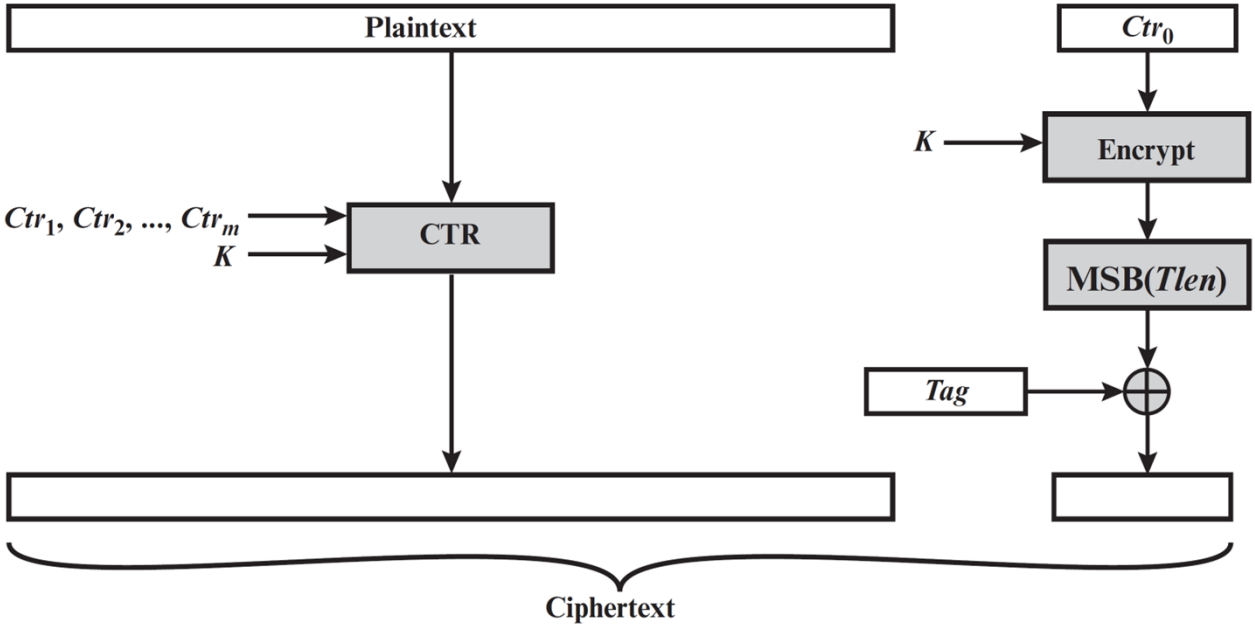
Counter with Cipher Block Chaining-Message Authentication Code (CCM)

- Variation of the encrypt-and-MAC approach (E+A)
- Originally developed for IEEE 802.11 Wi-Fi WPA security
- Standardized as NIST SP-800-38C
- Combines AES, CTR mode, and CMAC authentication
- 4 inputs
  - A single key K for both encryption and MAC
  - The plaintext message P (to be encrypted and authenticated)
  - Associated data A (only to be authenticated, e.g., protocol headers)
  - A unique nonce N that is associated to P and A (against replay attacks)

CCM authentication



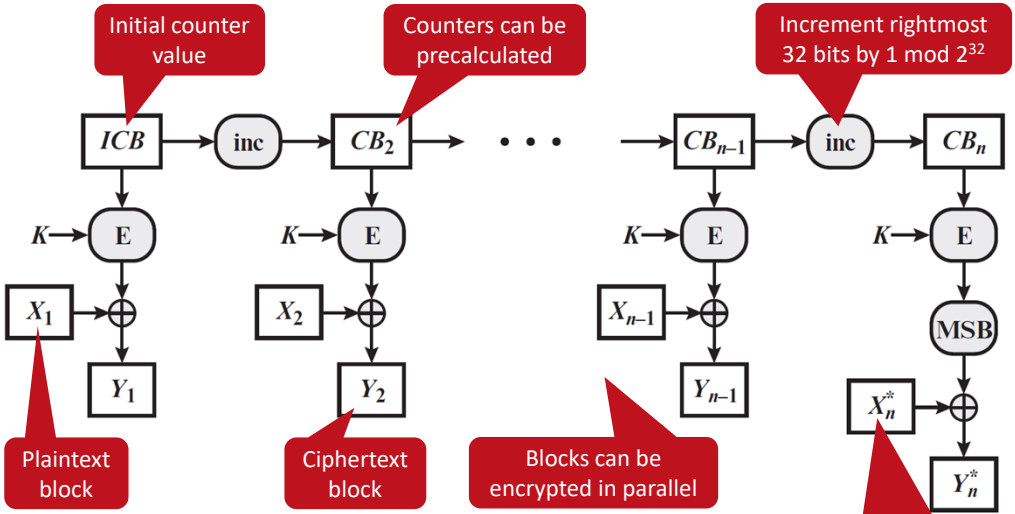
CCM encryption



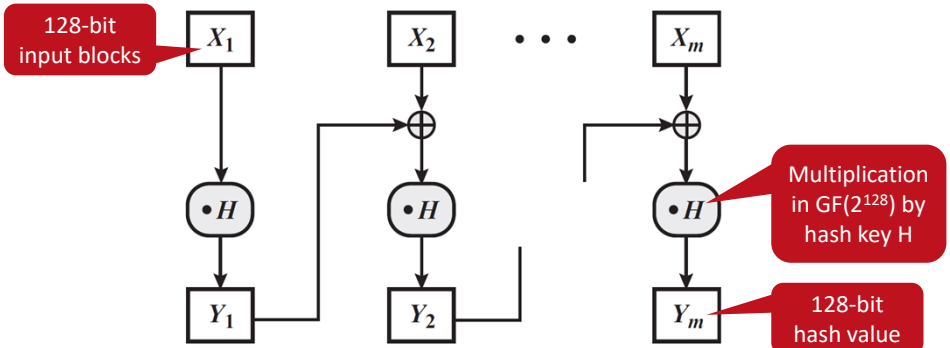
Galois/Counter Mode (GCM)

- Variation of the encrypt-then-MAC approach (E → A)
- Standardized by NIST in NIST SP 800-38D
- Designed for **efficiency**: high throughput with low cost and low latency
- Uses a variant of **CTR mode** that includes an authenticator tag (MAC)
  - GHASH: A keyed hash function to generate MAC
  - GCTR: CTR mode with counters determined by increment
- Authentication-only mode is also available, known as **GMAC**

GCM – GCTR


Initial counter value
Counters can be precalculated
Increment rightmost 32 bits by 1 mod 2^{32}
ICB
inc
CB_2
CB_{n-1}
inc
CB_n
K
E
X_1
Y_1
X_2
Y_2
X_{n-1}
Y_{n-1}
MSB
X_n^*
Y_n^*
Plaintext block
Ciphertext block
Blocks can be encrypted in parallel
Last block is not padded and can be shorter than block length

GCM – GHASH

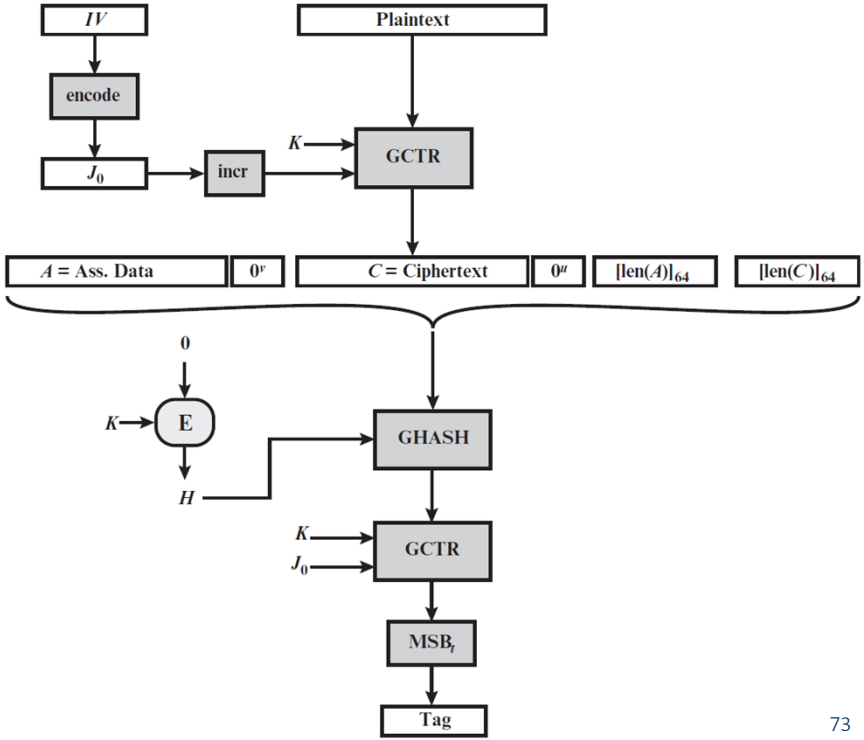


Powers of H can be precalculated

Blocks can be processed in parallel

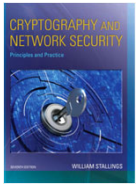
$$GHASH_H(X) = (X_1 \cdot H^m) \oplus (X_2 \cdot H^{m-1}) \oplus \cdots \oplus (X_{m-1} \cdot H^2) \oplus (X_m \cdot H)$$

Overview of GCM mode



Summary of message authentication

- Ensures integrity of transmitted data
- MACs combine a checksum, with the use of a secret key
  - Merkle-Damgård MAC of the form H(S|M) is vulnerable to length extension attack
- CMAC turns 3DES or AES into a MAC
- HMAC uses hash functions to provide a key-based cryptographic checksum
  - Generally more efficient than CMAC
- Authenticated encryption ensures confidentiality and integrity
  - CCM performs encrypt-and-MAC using AES, CTR mode, and CMAC
  - GCM performs encrypt-then-MAC using GHASH and GCTR mode



Link with the book

- Chapter 12 (Sections 12.1, 12.4—12.7)



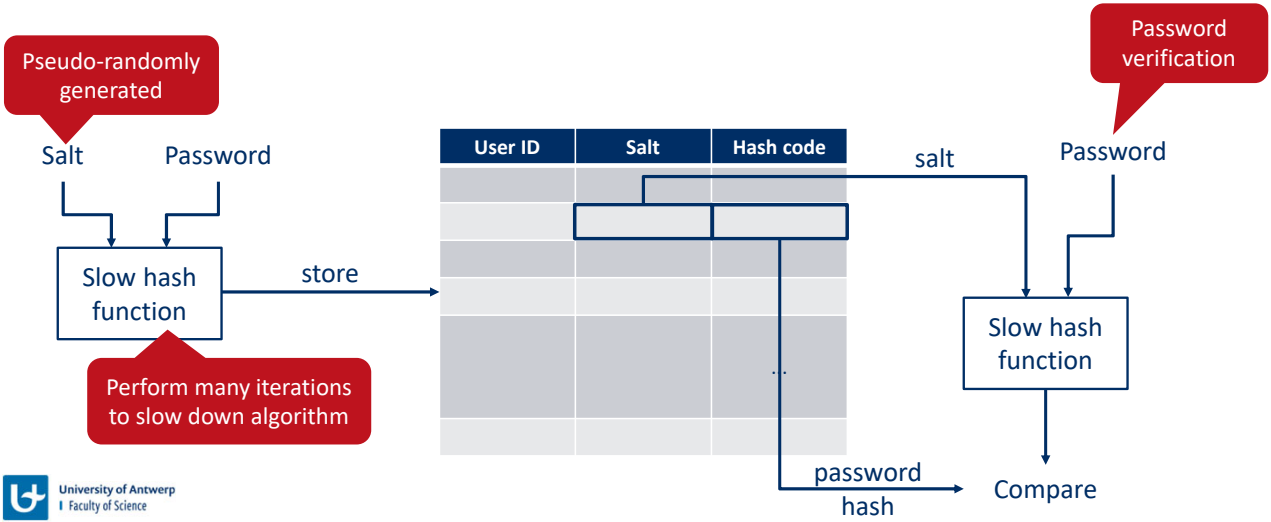
Link with the videos

- Week 3 (Message Integrity 1 & 2)
- Week 3 (HMAC)
- Week 4 (Authenticated Encryption 1 & 2)

Secure Password Storage

Password-based authentication

- Users log into a system using a combination of username (ID) and password
  - ID provides a unique identifier of the users for access control purposes
  - Password provides a way to authenticate the user

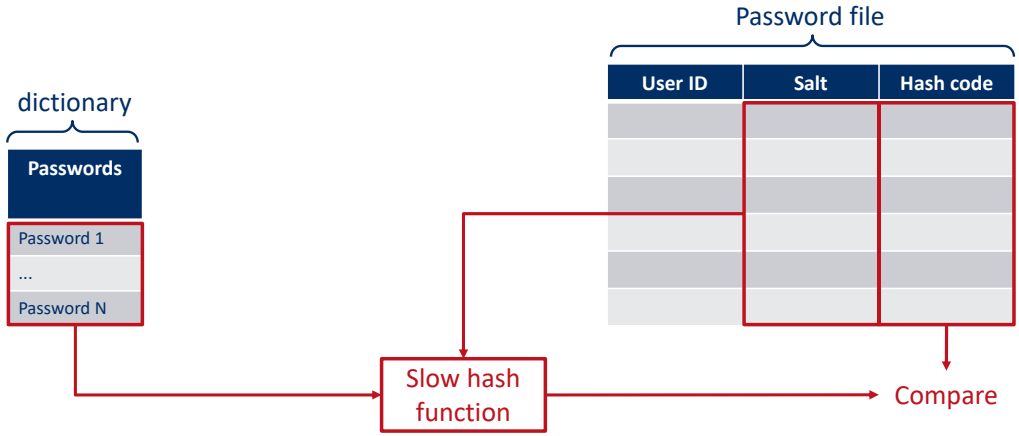


Purpose of a salt value

1. Prevents duplicate passwords from being visible in the password file
2. It greatly increases the difficulty of offline dictionary attacks
3. It becomes nearly impossible to find out if a person uses the same password on multiple systems

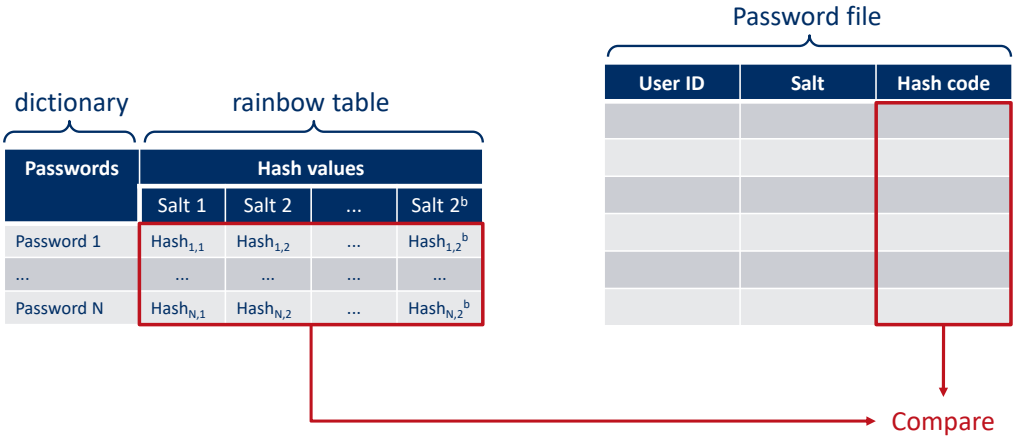
Password cracking: Dictionary attack

- Develop a large dictionary of possible passwords offline
- Try each dictionary entry hash against the password file
- Each possible password must be hashed with each available salt value



Password cracking: Rainbow table attack

- Trade-off time for space by precomputing potential hash values
- Hash values of each possible password with each possible salt are pre-calculated and stored in a file



When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

How much storage space is approximately required to store a rainbow table of 1 billion (10^9) 64-bit passwords hashed using SHA3-256 without salt?

0

- 40 megabytes
- 40 gigabytes
- 40 terabytes
- 40 petabytes

When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**

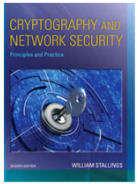
How much storage space would be required in total if a 16-bit salt was used?

0

- 2 megabytes
- 2 gigabytes
- 2 terabytes
- 2 petabytes

Summary of secure password storage

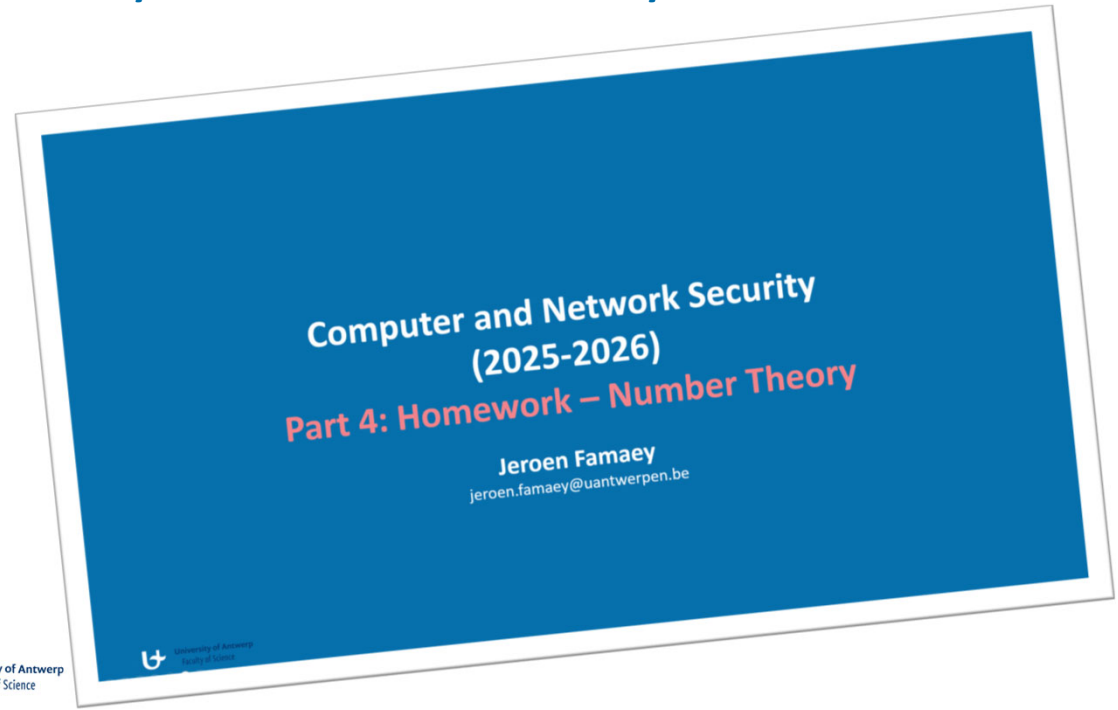
- User ID provides unique identification and access control
- Password provides authentication of the user
- Stored password as hashed with a salt to keep them secret
  - A slow hash function is used to slow down brute force password guessing attacks
  - Salt is used to increase difficulty of offline guessing attacks
- Two main types of attacks
  - Dictionary attack: List of common passwords that are tested online
  - Rainbow table attack: Salted hashes are pre-calculated to trade off time for storage space



Link with the book (online chapters)

- Chapter 22 (Section 22.3)

Homework by next week: Number Theory



End of Part 3